

**AI Course**

# **Capstone Project Final Report**

**For students (instructor review required)**

© 2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of this document.

This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.

To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

< Project Title >

/ /

Cogni

Marwan Elamami (Team Leader)  
Abdulrahman Khengari  
Ahmed Alaeb  
Sanad Ali  
Mohammed Thabet, Abdul Majid

# Content

## **1. Introduction**

- 1.1. Background Information
- 1.2. Motivation and Objective
- 1.3. Members and Role Assignments
- 1.4. Schedule and Milestones

## **2. Project Execution**

- 2.1. Data Acquisition
- 2.2. Training Methodology
- 2.3. Workflow
- 2.4. System Diagram



## **3. Results**

- 3.1. Data Preprocessing
- 3.2. Exploratory Data Analysis (EDA)
- 3.3. Modeling
- 3.4. User Interface
- 3.5. Testing and Improvements

## **4. Projected Impact**

- 4.1. Accomplishments and Benefits
- 4.2. Future Improvements

## **5. Team Member Review and Comment**

## **6. Instructor Review and Comment**

# 1. Introduction

## 1.1. Background Information

Dyslexia affects reading through phonological processing, and Modern Standard Arabic adds barriers that tools built for Latin scripts do not address.

Short vowels are normally omitted in written Arabic, so one written form can carry several readings: كُتِبَ is kataba (he wrote), kutiba (it was written) or kutub (books). The reader must resolve this from context, which is exactly the process dyslexia impairs. Arabic is also templatic and agglutinative: conjunctions, articles and object pronouns attach directly to the stem, producing long unbroken words that crowd the line. Prose chains clauses with و and ف across long stretches without sentence-final punctuation, so a single sentence can exceed working memory.

Full diacritization is not a complete answer either: marking every character adds visual clutter of its own. The useful intervention is to simplify the text first, then vocalise it.

We found no deployed Arabic dyslexia-support software, and no large open Arabic simplification corpus, which shapes both the product and the data work below.

## 1.2. Motivation and Objective

Objective: let an Arabic reader with dyslexia read difficult text independently, wherever they meet it.

The product is an Android plugin registered on the system PROCESS\_TEXT intent, so “تبسيط” (Simplify) appears in the text-selection menu of every app. The reader selects text in a message, a news page or a PDF and is helped in place — no copying, no pasting, no separate app to open. A standalone app was rejected for this reason: it reintroduces the friction the project exists to remove.

Every model runs on the device. The text never leaves the phone, the tool works offline, and nothing depends on a hosted service during the demo.

The pipeline has three stages: simplify, then diacritize, then read aloud with synchronized word highlighting.

## 1.3. Members and Role Assignments

Marwan Elamami — Team Leader and On-Device Model Lead. Project management, system architecture, model compression and quantization, integration, human evaluation.

Abdulrahman Khengari — Data Lead. Simplification corpus, readability classifier, annotation guidelines and annotator agreement.

Ahmed Alaeb — Model Training Lead. Training and evaluating the simplification models, corpus conversion, training infrastructure.

Sanad Ali — Machine Learning Engineer. Diacritization model selection and benchmarking, co-training the simplification models.

Mohammed Thabet — Application Engineer. Android plugin, on-device inference integration, read-aloud with word highlighting.

Abdul Majid — Evaluation and Benchmarking Lead. Test-set design and locking, leakage detection, scoring pipeline, automatic and human evaluation.

## 1.4. Schedule and Milestones

Week 1 (16–20 Sep) — complete: data pipeline and validators; readability classifier; test-set design; product-form decision; application skeleton.

Week 2 (21–27 Sep) — in progress: corpus generation and review; first and second simplification training runs; diacritization benchmark; locked test sets and scoring pipeline; Android plugin with on-device inference.

Week 3 (28 Sep – 4 Oct): architecture frozen; int8 quantization with size and latency measured on a real device; annotator agreement reported; full benchmark including human error annotation; human evaluation with dyslexic readers; end-to-end integration.

Week 4 (5–6 Oct): final report and demo video for submission on 5 October; final pitch on 6 October.

Work is tracked package by package in the Work Breakdown Structure: 8 phases, 38 work packages, one named owner each.

## 2. Project Execution

### 2.1. Data Acquisition

Five sources, all obtained under licences that permit our use. Counts are of usable pairs after filtering.

BAREC (CAMEL Lab, CC BY-SA 4.0) — 69,441 sentences with 19-level readability labels. Source sentences to simplify, and the training labels for the readability classifier.

SAMER Simplification Corpus v1 (CAMEL Lab) — 14,343 complex-to-simple pairs at three levels. CAMEL Lab gave written approval to fine-tune on it and publish the resulting weights non-commercially; the corpus itself is never redistributed.

Baseet (University of Jeddah) — 34,294 sentences with three LLM-generated simplification levels, built from BAREC and SAMER. 3,190 rows (9.3%) overlapped our locked test sets and were removed before any training use, leaving 31,104.

DAASI (CC0) — 1,509 human-written pairs of government and insurance text, plus a 350-pair held-out test set. This is our only out-of-domain test set with human-written references.

Bayan corpus (built by the team) — LLM-generated pairs behind a human-reviewed quality gate, with the source text, judge score, reasoning and readability label retained for each accepted pair.

Tashkeela provides fully diacritized Arabic for validating the diacritization stage.

### 2.2. Training Methodology

Model 1 (measured). AraT5v2-base-1024, 368M parameters, fine-tuned on SAMER L5L3.

Learning rate  $1e-4$ , AdamW with weight decay 0.01, 100 warmup steps, effective batch 16, fp16, gradient checkpointing, 10 epochs / 3,250 steps, seed 42. Identity pairs capped at 20% in training and left untouched in dev and test.

Three corrections follow from its measured behaviour, and apply to every later run:

Checkpoint selection on SARI over the changed-reference subset, decoded greedily. Selecting on full-dev SARI rewards a model that does nothing, because roughly half the dev set needs no change; selecting under beam search measures a decoder we do not ship.

Early stopping on validation loss rather than SARI. Validation loss bottomed near step 2,200 while SARI wandered inside noise for a further 1,400 steps and chose a checkpoint on a 0.18 difference.

Training on plain and diacritized copies of every pair. SAMER carries no diacritics at all, and the resulting model degrades badly on diacritized input.

Model 2 (planned). The same recipe on a mixed corpus of roughly 107,000 pairs across three domains, with level tokens so one model offers three simplification strengths plus a leave-unchanged mode. AraBART (139M) is trained alongside AraT5v2 on identical data; the choice between them will be made on measured results, with the smaller model preferred at equal quality because it makes on-device decoding cheaper.

### 2.3. Workflow

**Data.** Hard sentences are drawn from BAREC and SAMER. An LLM generates candidate simplifications; an LLM judge calibrated on human annotations scores meaning preservation; a fine-tuned Arabic BERT classifier checks that the simplified side is genuinely easier; a human reviewer approves or rejects. Sentences that are already easy, and religious scripture and poetry, are kept unchanged and used as leave-unchanged examples.

**Training.** Accepted pairs are converted to one shared nine-column format, checked for leakage against the locked test sets, and only then used for fine-tuning.

**Evaluation.** Predictions are scored against a copy-the-input baseline on the same split. Only the Evaluation Lead runs the locked test sets, and only on final models; everyone else tunes on the dev split.

**Inference on the device.** Selected text is stripped of diacritics, simplified, re-diacritized by Libtashkeel, and passed to the read-aloud stage. The order matters: the simplifier is trained on undiacritized text, so diacritization must come after it, never before.

### 2.4. System Design

A single Android application containing two activities and three models.

**ProcessTextActivity** registers `android.intent.action.PROCESS_TEXT` with `contentType` `text/plain` and the label "تبسيط". It uses a translucent theme so the result floats over the host application, which avoids requesting the `SYSTEM_ALERT_WINDOW` overlay permission entirely. The result is shown in a Material 3 modal bottom sheet.

**MainActivity** is a launcher with a paste box and a model-status card, used for demonstrating the pipeline without a third-party app.

Inference runs through ONNX Runtime Mobile for every model, behind a `TextSimplifier` interface so the engine can be swapped without touching the user interface. Decoding is greedy, and the tokenizer is exported inside the ONNX graph so no Python runtime is needed on the phone.

**Size budget:** 250 MB for the simplifier, plus about 45 MB for diacritization and the read-aloud model. Vocabulary-pruned int8 is approximately 223 MB and floats; int4 (approximately 112 MB) is the fallback if load time or memory on a mid-range phone proves to be the constraint. Models are downloaded on first run rather than bundled, which keeps the install small and allows the model to be replaced without shipping a new build.

## 3. Results

### 3.1. Data Preprocessing

Normalisation for comparison only. Diacritics and punctuation are stripped and alef variants (إ, إ, ؤ, and ي/ي) are unified. This normalisation is used for leakage detection and scoring alignment; it is never applied to training text, where the original orthography is preserved.

**Test sets locked first.** Two splits were locked before any training and are never trained on: BAREC test (7,286 rows) and SAMER test (3,277 rows). Each is recorded

by row count and SHA-256 in a committed manifest, so any team member can verify they hold identical data. Both are the official upstream splits, unmodified.

Leakage filtering (measured). Every training file is checked against both locked sets for exact matches and near-duplicates, using character 5-gram Jaccard overlap at a 0.8 threshold. Matches shorter than five words are reported but not treated as leakage, because short form ubiquitous fragments recur across any corpus.

This check found that 3,190 of Baseet's 34,294 rows (9.3%) overlapped our locked test sets: 1,829 rows touching SAMER test and 1,361 touching BAREC test. Put the other way, 1,835 of SAMER's 3,933 substantive test sentences (47%) appear somewhere in that corpus. Those rows were removed and the filtered corpus re-verified clean before any training use.

Identity pairs. Pairs whose simplified side equals the source are capped in training but kept in full in dev and test, because leaving easy text alone is part of the task and a model that cannot do it is not usable in a plugin that receives arbitrary selected text.

### 3.2. Exploratory Data Analysis (EDA)

The corpora differ in ways that turned out to drive model behaviour, so they are reported here rather than assumed.

SAMER is literary. Its text is drawn from early twentieth-century novels; the test split has a median sentence length of 6 words, and 17.8% of its rows are three words or fewer — chapter headings, dialogue fragments and exclamations rather than sentences. 48.8% of test rows are identity pairs, where the human simplifier changed nothing.

SAMER splits by chapter, not by novel, so all fifteen novels appear in all three splits. Its test set is therefore fully in-domain and SARI measured on it reads optimistically. This is why a second, out-of-domain test set is necessary.

SAMER contains no diacritics at all (0% of rows), while 56% of BAREC's sentences carry them.

BAREC duplicates itself: 311 sentences appear in both its train and test splits, and 797 of its 7,286 test rows are two words or fewer. Both facts cause false alarms in leakage checking and are the reason the check has a minimum-length floor.

Baseet is general domain and closer to the product: median source length 14 words, 35.4% diacritized, and no identity pairs at any level. Its strongest simplification level shortens a 14-word median source to 9 words.

DAASI is government and insurance text, median 12 words, 32% diacritized, no identity pairs.

### 3.3. Modeling

Copy-the-input baseline (measured). Reported first, because on a corpus where roughly half the sentences need no change a model that does nothing can score well. On SAMER test the baseline scores SARI 77.5 overall, but 56.1 on the 1,678 rows a human actually simplified. The second number is the bar any model must clear. On BAREC, which has no reference simplifications, only the reference-free metrics apply.

Model 1 results (measured). On SAMER dev the model scores SARI 77.06 overall and 63.66 on changed references, against a baseline of 74.25 and 56.21 on the same rows — a gain of +2.81 overall and +7.46 where the work is. Under greedy decoding, which is what the on-device build uses, the same model scores below the baseline. Beam search accounts for the difference, and the decision of whether to pay its cost on the device is pending a latency measurement.

Behavioural findings (measured). The model copies its input . . . % of the time against a human rate of 41.2%, and returns unchanged text on 46.8% of the rows a human simplified. Where it does edit, 75.3% of edits change a single word, against 53.2% for the human

references, and the median prediction-to-source length ratio is 1.000 — it substitutes vocabulary but never restructures a sentence.

Out-of-domain behaviour degrades sharply. On diacritized BAREC input, which the training data never contained, only 60% of source words survive into the output and 38% of sentences lose more than half their content to character-level corruption; on undiacritized input the figures are 90% and 3%. Diacritics must therefore be stripped before the simplifier.

Meaning errors are not rare: 832 of the model's substitutions disagree with the human reference, 347 of them on words the human deliberately left alone. SARI and BLEU do not detect this class of error, which is why BERTScore and human error annotation are part of the evaluation.

Diacritization (measured). Four models were benchmarked on the pinned Tashkeela test split. Libtashkeel was selected: DER 6.91% excluding case endings, 4.79 MB, 125 sentences/sec, MIT licence, exports to ONNX. CATT is statistically tied on accuracy but sixteen times larger and six times slower. CAMEL Tools was rejected on a stronger ground than accuracy: it altered base letters in 32% of sentences, corrupting the text before the simplifier sees it.

### 3.4. User Interface

Selecting Arabic text in any application shows “تبسيط” in the system text-selection menu. Choosing it opens a modal bottom sheet over the host application showing the original text in a muted card and the simplified text below it, with controls for read-aloud, copy and settings.

Typography is set for dyslexic reading rather than density: line height 1.6, additional line spacing of 10sp, letter spacing 0.04, and right-to-left text left unjustified, because justification in Arabic stretches words and creates uneven rivers of space. The sheet sits in a scrolling container so long selections are never clipped.

Text selection behaviour was verified across Chrome, WhatsApp, Google Keep, Gmail and PDF readers, and the differences recorded in a compatibility matrix.

### 3.5. Testing and Improvements

Application. 41 unit and Robolectric tests cover intent extraction, empty-input handling, view binding and the dyslexia typography attributes.

Evaluation tooling. The leakage checker is tested against a deliberately planted duplicate and must exit non-zero. The scoring script is exercised end to end on real data rather than only on fixtures; doing so exposed a bug that placed every BAREC row in the unchanged-reference bucket instead of a separate no-reference bucket, since BAREC has no gold simplifications at all.

Improvements already made as a result of measurement: the copy baseline is now reported beside every score; checkpoint selection moved to the changed-reference subset under greedy decoding; diacritics are stripped before the simplifier; and 9.3% of a published corpus was removed before it could contaminate our results.

## 4. Projected Impact

### 4.1. Accomplishments and Benefits

Measured to date: a copy-the-input baseline on two locked test sets; a first simplification model trained and scored against it, with its failure modes characterised; a four-model diacritization benchmark; and 9.3% test-set contamination detected and removed from a published Arabic simplification corpus before training.

Built: a working Android plugin shell on the PROCESS\_TEXT intent; a locked and hashed test-set manifest; a reusable leakage checker; and a scoring pipeline that reports a copy baseline beside every metric.

Benefits. Greater reading independence for Arabic readers with dyslexia, at the moment and place the difficulty occurs rather than in a separate application. Support for students, teachers and schools, and for readers with low literacy and learners of Arabic. Everything runs on the device, so the tool works off-line and the reader's text stays private.

For the field: an Arabic simplification corpus, an evaluation benchmark with human-written references, and a leakage-detection tool, none of which existed in openly reusable form.

#### 4.2. Future Improvements

Immediate, before submission: complete the mixed-corpus training runs and choose between AraT5v2 and AraBART on measured results; quantize the winner to int8 and measure size and latency on a real phone; run the human evaluation with Arabic readers who have dyslexia; report annotator agreement on the corpus.

Known limitations to carry forward. The current model substitutes vocabulary but does not restructure sentences, and clause splitting is arguably the larger part of the problem for dyslexic readers. Automatic metrics do not detect meaning-altering substitutions, so human annotation remains necessary. On-device latency is not yet measured, so the choice between greedy and beam decoding is still open.

Beyond the capstone: fine-tune the diacritizer on simplified output rather than using it unchanged; add the read-aloud stage with word-level timing; and expand the corpus beyond the three domains currently covered.

### 5. Team Member Review and Comment

Each member writes their own review in the table below before submission.

<ATTACH A TEAM PICTURE HERE>

| NAME | REVIEW and COMMENT |
|------|--------------------|
|      |                    |
|      |                    |
|      |                    |
|      |                    |
|      |                    |

### 6. Instructor Review and Comment

| CATEGORY | SCORE | REVIEW and COMMENT |
|----------|-------|--------------------|
|----------|-------|--------------------|

|                                      |        |  |
|--------------------------------------|--------|--|
| <b>IDEA</b>                          | __/10  |  |
| <b>APPLICATION</b>                   | __/30  |  |
| <b>RESULT</b>                        | __/30  |  |
| <b>PROJECT<br/>MANAGEMENT</b>        | __/10  |  |
| <b>PRESENTATION<br/>&amp; REPORT</b> | __/20  |  |
| <b>TOTAL</b>                         | __/100 |  |